

WEEK 3 — LISTS, FUNCTIONS & CSV

DHT11 Sensor | Data Science with ESP32 | Grade 8

1 LISTS

```
# Empty list
temps = []
# Add item
temps.append(28.5)
# First / last item
print(temps[0])
print(temps[-1])
# Count
print(len(temps))
# Last 30 (for graph)
last30 = temps[-30:]
# Loop through
for t in temps: print(t)
```

2 LIST METHODS

.append(val) Add to end

.pop() Remove last

len(list) Count items

list[-30:] Last 30

list[0] First item

list[-1] Last item

3 FUNCTIONS

```
# Define function
def find_port():
    for port in comports():
        if "CH340" in port.desc:
            return port.device
    return None

# With parameter
def get_feels(temp):
    if temp > 35: return "Hot"
    elif temp > 25: return "Warm"
    else: return "Cool"

# Call functions
PORT = find_port()
level = get_feels(28.5)
```

4 CSV WRITING

```
import csv, os

FILE = "dht11_data.csv"

# Create with header
if not os.path.exists(FILE):
    with open(FILE, "w", newline="") as f:
        csv.writer(f).writerow(
            ["time", "temp", "hum", "feels"])

# Append a row
with open(FILE, "a", newline="") as f:
    csv.writer(f).writerow(
        [now, temp, hum, feels])

# "w"=create "a"=append
```

5 FULL DHT11 SCRIPT

```
import serial, csv, os
import serial.tools.list_ports
from datetime import datetime

FILE="dht11_data.csv"
temps, humids, times=[], [], []

def find_port():
    for p in list_ports.comports():
        if "CH340" in p.description:
            return p.device
    return None

def get_feels(t):
    return "Hot" if t>35 else "Warm" if t>25 else "Cool"

PORT=find_port()
ser=serial.Serial(PORT, 9600, timeout=1)

while True:
    line=ser.readline().decode("utf-8", errors="ignore").strip()
    if "Humidity" in line and "Temperature" in line:
        try:
            parts=line.split(",")
            h=float(parts[0].split(":")[1])
            t=float(parts[1].split(":")[1])
            now=datetime.now().strftime("%H:%M:%S")
            times.append(now)
            temps.append(t)
            humids.append(h)
            with open(FILE, "a", newline="") as f:
```

FUNCTION RULES

def Keyword to define function

name(): Descriptive name + colon

return Send value back to caller

4 spaces Indent ALL code inside

Call PORT = find_port()

COMMON ERRORS

IndexError: list index

Fix: Check len(list)>0 first

SyntaxError: expected":

Fix: Add colon after def/if/while

FileNotFoundError

Fix: Use "a" mode not "r" mode

TypeError: str+float

Fix: Use f-string: f"{val}"

GOLDEN RULES:

1. Upload matching Arduino code FIRST
2. Close Arduino IDE before running Python
3. Check len(list)>0 before accessing list[0]